

LA ROCHELLE UNIVERSITÉ

LICENSE 3 INFORMATIQUE



Computer Vision & Artificial Intelligence

Project Report: Chess Game Video Analysis

Professors MASCARILLA Laurent
PETERI Renaud

Members TRAN Minh Duong

Abstract

This project involves developing a Python application using OpenCV to automatically analyze and transcribe chess game from a top-down video stream. The system is divided into several computer vision tasks: detecting and rectifying the chessboard, tracking player hand movements to determine when a move has been completed, and identifying the position and color of chess pieces. The final output is a full transcription of the game in algebraic notation.

Introduction

Chess is a popular strategy game that demands rigorous logical analysis. In the context of computer vision, automating the transcription of a physical game presents a complex challenge: transforming a continuous 3D video stream into digital data.

By using a top-down camera feed and OpenCV, this project aims to bridge the gap between the physical board and digital notation. By leveraging image processing techniques such as perspective transformation, motion analysis,... we can create an autonomous system capable of “reading” the game in real-time. This automation eliminates human error in recording moves and allows for immediate digital analysis using algebraic notation.

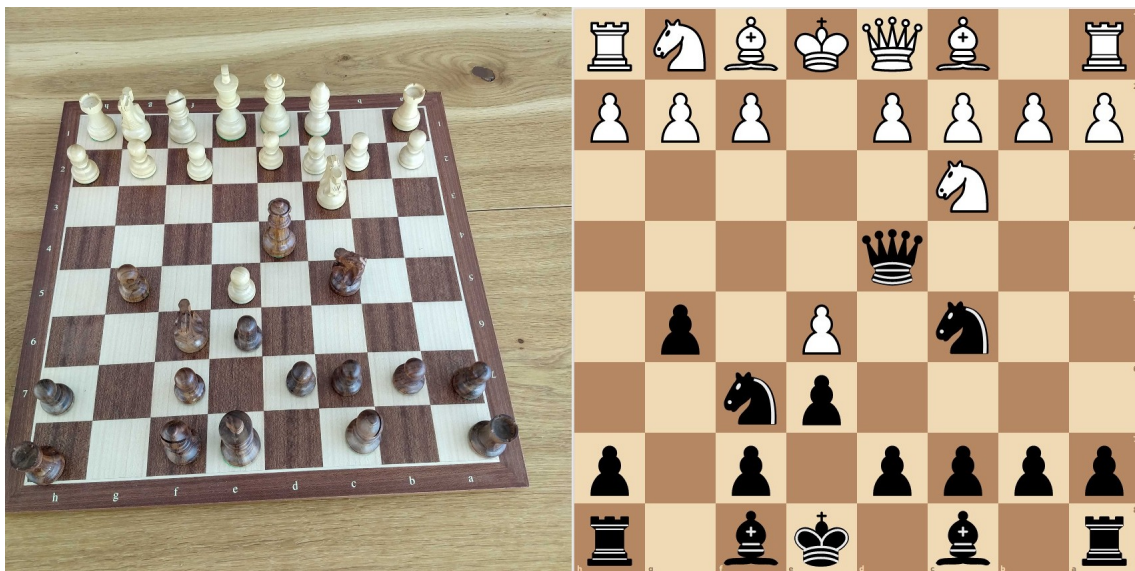


Figure 1: Converting a physical chessboard into a digital one

I. Board Detection and Rectification

I.1. Objective

The primary goal of this part is to extract the 8×8 chessboard from the raw video feed, then transform it into a top-down square image. This helps eliminate perspective distortion and background noise, such as the table, tripod or player movements, which could otherwise interfere with the analysis.

I.2. Methodology

The pipeline for board rectification is divided into three main processes:

A. Image Preprocessing - Edges Detection

To extract the board structure, the frame is first converted to grayscale and smoothed using a **Gaussian Blur** with a 3×3 kernel to filter out high-frequency noise. We then apply a **Canny Edge Detector** to identify the grid boundaries. Finally, a **Morphological Closure** (3×3 rect kernel) is performed to fill small gaps in the grid lines, ensuring a continuous contour regardless of lighting conditions.

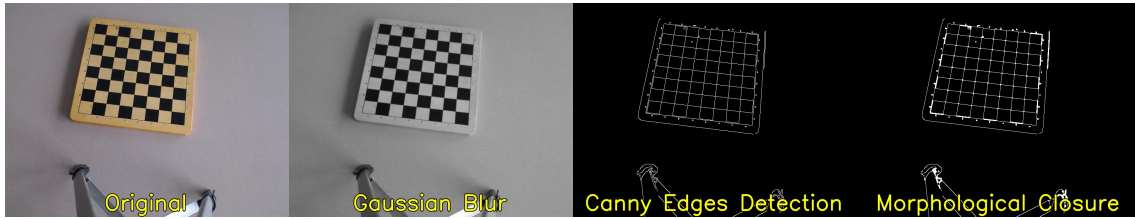


Figure 2: Preprocessing Workflow: from the raw input to the closed-edge map

B. Corner Extraction

The system identifies the chessboard by finding the largest external contour in the edge map, assuming the 8×8 grid is the most suitable geometric feature. To extract the four specific corners from this contour, we use the following coordinate extremes:

- **Top-Left:** Point with the minimum $(x + y)$ sum.
- **Bottom-Right:** Point with the maximum $(x + y)$ sum.
- **Top-Right:** Point with the minimum $(y - x)$ difference.
- **Bottom-Left:** Point with the maximum $(y - x)$ difference.

C. Perspective Transformation

Once the corners are localized, a **Homography Matrix** is calculated using OpenCV’s `getPerspectiveTransform`. This matrix maps the distorted coordinates to a perfect 400×400 pixel square. We then apply `warpPerspective` to generate the rectified “top-down” view ready for game transcription.

I.3. Results and Discussion

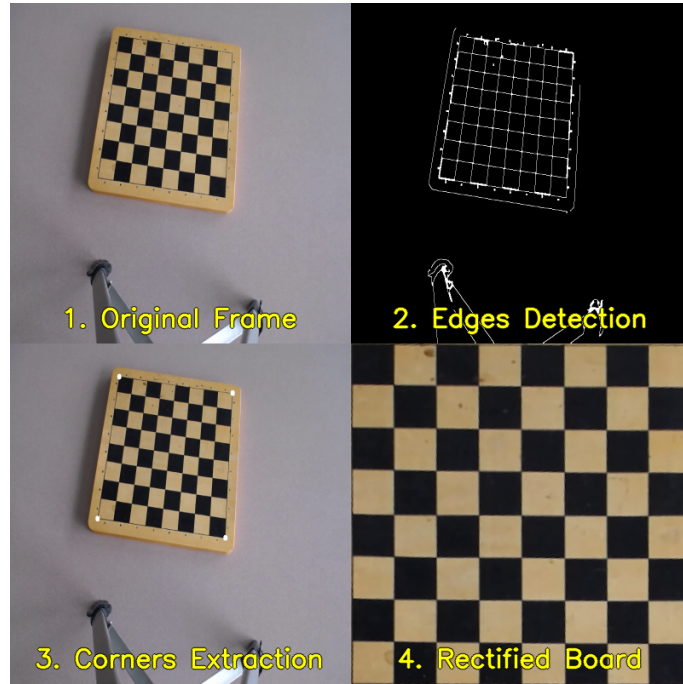


Figure 3: Board Extraction Result

The implementation successfully generates a clean, standardized view of the board. By extracting these corner coordinates from an empty-board video and storing them into `corners_coordinates.json`, we have essentially “**locked**” the chessboard so that the system can remain stable throughout the game.

While the rectified image simplifies determining cell occupancy, it is noted that the vertical shape of pieces may be distorted. Consequently, while the rectified board is used for coordinate mapping, the *original frame* may be referenced for piece identification to preserve visual features.

II. Player Hand/Move Detection

II.1. Objective

The objective of this part is to identify precisely *when* a player’s hand enters and leaves the chessboard. This helps us know when a move is made.

II.2. Methodology

For this, we chose the **Background Subtraction** method using OpenCV’s **MOG2** algorithm. This approach is more robust than simple frame differencing because it builds a statistical model of the static background (the empty board). A player hand that deviates significantly from this model is identified as “foreground.”

Here’s how it works:

1. **Background Learning:** The system constantly observes the rectified chess-board (from Part 1) and learns what the “normal”, static board looks like.
2. **Motion Mask Generation:** When a hand enters the frame, the background subtractor compares the current image to its learned background model. It generates a “motion mask” – a grayscale image where white pixels indicate areas of movement (the hand), and black pixels represent the static background. (To ensure a clean mask, we configured the subtractor to `detectShadows=False`, which prevents shadows from being mistaken as motion.)
3. **Contour Detection:** We then look for large contours in this motion mask. Small contours are ignored (`min_area = 1000`) to filter out minor noise.
4. **State Machine:** A simple state machine is used to track the interaction:
 - **Board is Stable:** No hands are detected.
 - **Hand Detected:** A hand (large contour) has entered the frame.
 - **Analyzing the Move:** The hand has left the frame, returning the board to a stable state.

It’s important to note that this part focuses solely on detecting the *presence* and *absence* of a hand. Determining *which player* made the move will be handled in Part 3, by comparing the board state before and after the hand intervention.

II.3. Results and Discussion

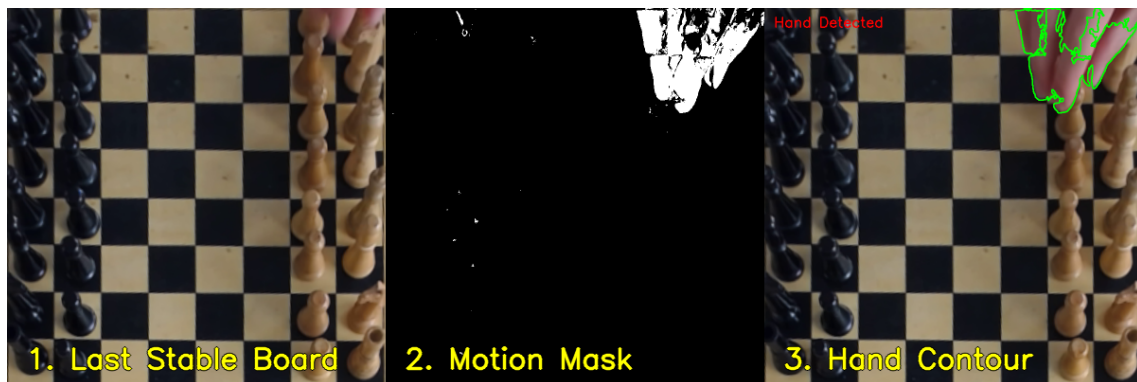


Figure 4: Hand Movement Detection Pipeline Summary

- **Last Stable Board:** The rectified board just before the hand entered.
- **Motion Mask:** The binary mask generated by the background subtractor, clearly showing the detected hand in white.
- **Hand Contour Detected:** The final processed image, showing the original board with the detected hand outlined in green.

We have successfully implemented the detection of player hand movements via background subtraction. This allows us to know exactly when a move is being made, which is crucial for the next step of identifying the move itself.

III. Pieces Detection and Identification

III.1. Objective

The objective of this part is to accurately detect and identify the chess pieces on the board. This is essential for understanding the game state and the specific move being made.

III.2. Methodology

III.3. Results and Discussion